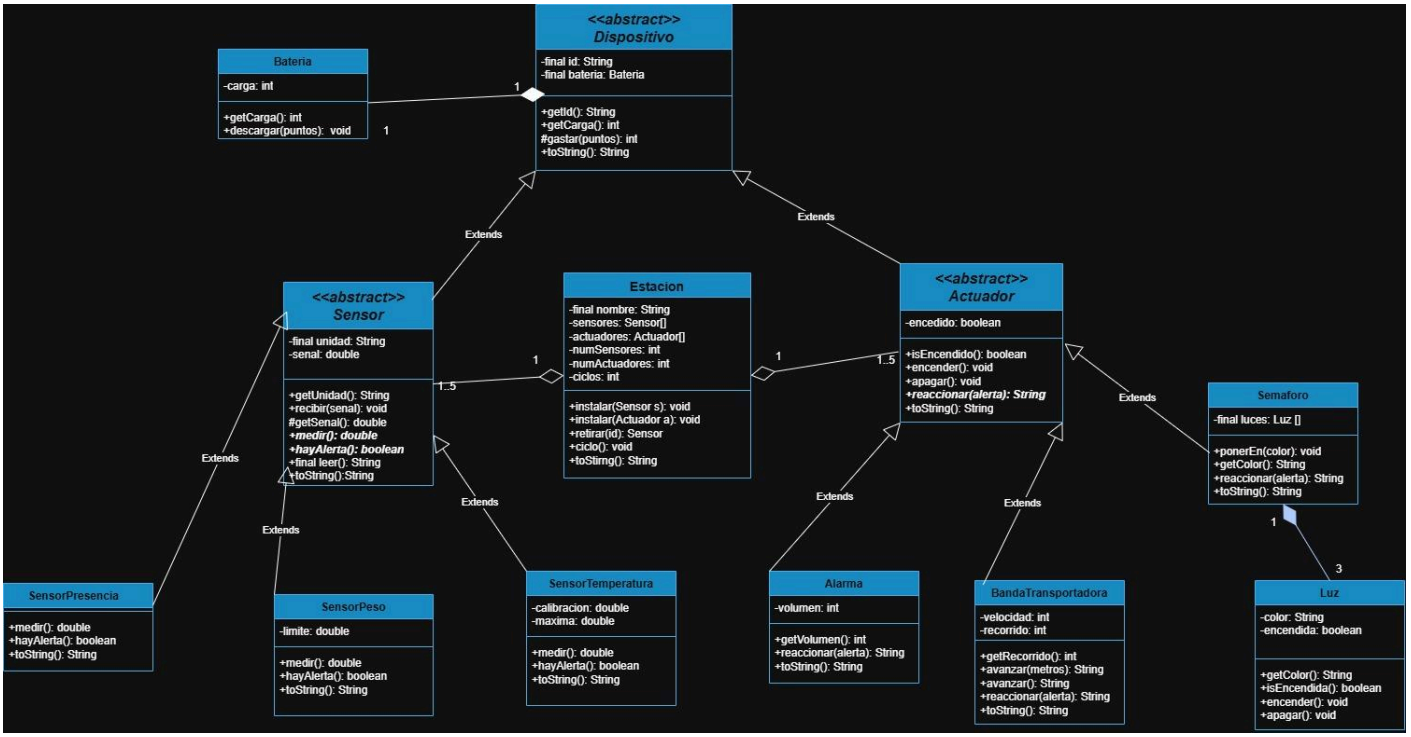
Practica de laboratorio 8:
HERENCIA - SEGUNDA PARTE

Hernández Aviña Karen Itzel
Hernández González Mauricio

DIAGRAMA



CAPTURA DE SALIDA

```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hot
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: ActuadorA1 (100%) banda , 2 m por ciclo
Instalado: ActuadorA2 (100%) semaforo en apagado
Instalado: ActuadorA3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 grados
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 grados
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
```

```
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 grados ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos

Process finished with exit code 0
```

Practica de laboratorio 8:
HERENCIA - SEGUNDA PARTE

1. Sobrecarga contra sobrescritura: un ejemplo de cada una sacado de tu código, y la diferencia.

Sobrecarga:

```
public Actuador(String id, int carga){
   super(id,carga);
}
public Actuador(String id){
   this(id,100);
}
```

Sobrescritura:

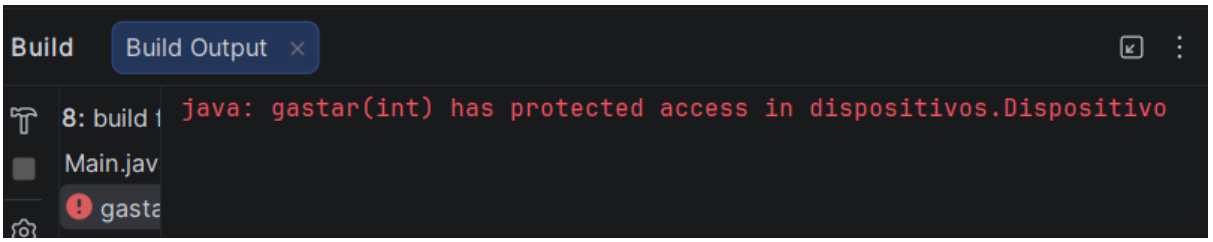
```
@Override
public String toString(){
   return "Actuador"+ super.toString();
}
```

La sobrescritura nos permite cambiar el comportamiento del método de la clase padre, puede heredar ese comportamiento y puede agregar más cosas, al poner @Override le pedimos a javac que confirme que estamos sobreescribiendo el método. La sobrecarga se puede usar tanto en métodos como en constructores,consiste en poner el mismo nombre en varios métodos o constructores pero diferentes parámetros.

2.Composición contra agregación: ¿por qué la Bateria es composición y el sensor en la Estacion es agregación? Piensa en quién hace el new y en qué le pasó a S2 en el ciclo 3.

Bateria es composición ya que no tiene sentido que exista si no hay una Estacion, con new se crea un nuevo objeto,es decir Estacion crea una nueva Batería,un sensor puede cambiar de estación y seguiría funcionando igual,o en el caso de lo que pasó en el ciclo 3 se retiró el S2,pero después se nos indica que lo volvieron a instalar,y eso no afecta en la ejecución de Estacion.

3. Modificadores: agrega en Main la línea bascula.gastar(5); , copia el error y explica por qué SensorPeso sí puede usar gastar y Main no. Bórrala antes de entregar.



En la clase dispositivo el método **gastar(int puntos)** tiene un modificador de acceso protected, lo cual quiere decir que se puede acceder a este método en su clase, y que sus hijas también puede acceder, como SensorPeso es hija de Sensor,y Sensor a su vez es hija de Dispositivo, puede acceder a este método,pero Main no es hija,por eso no puede acceder